

FAI 모듈 리팩토링

개발자 가이드

대상: FAI 코드를 수정하거나 새 모델을 추가하는 개발자

목표: 기존 방식과 새 방식의 차이를 이해하고, 실무에서 바로 적용

참고

- 본 슬라이드는 마크다운 파일을 Marp 로 변환한 것입니다.
 - 참고 링크1 [Marp 공식 사이트](#)
 - 참고 링크2 [기술 블로그](#)

목차

슬라이드 제목
왜 리팩토링했나?
폴더 구조 변화
핵심 신규 파일
아키텍처: Strategy + Factory 패턴
코드 비교 ①: 측정 데이터 접근
코드 비교 ②: FAI Use 플래그
1. 새 모델 추가 체크리스트
2. 신규 모델 등록: ModelTypeList.h + ModelNameCode.ini
새 Inspector 클래스 만들기 (CHS 모델 예시)
팩토리 등록 방법 (모델 전환 코드)
지원 모델 현황
동적 FAI 측정 엔진 (별도 문서 참조)
정리 및 Q&A

왜 리팩토링했나?

기존 코드의 문제점

1. 데이터가 앱 전역에 산재

- THEAPP.m_StructFaiMeasure , THEAPP.m_bFAIUse 등이 Algorithm.cpp 곳곳에 직접 노출
- FAI 관련 로직을 찾으려면 800KB짜리 Algorithm.cpp 를 전체 탐색해야 함

2. 매직 넘버 남발

```
// 이게 무슨 의미인지 즉시 알기 어려움
if (THEAPP.m_bFAIUse[5]) { ... }
m_pStructFAI->m_dFAIMeasureValue[14][0] = dResult;
```

3. 거대 파일 의존 → 변경 시 충돌 위험

- 모든 모델(BOI/AKC/ACT/ATW)의 FAI 로직이 Algorithm.cpp 하나에 혼재
- 한 모델 수정 시 다른 모델 코드에 영향 우려

폴더 구조 변화

기존: FAI/ 폴더 없음

```
Algorithm.cpp ← 800KB
├── BOI FAI 로직
├── AKC FAI 로직
├── ACT FAI 로직
├── ATW FAI 로직

AlgorithmThreadDefine.h
├── CenterlineMeasureStruct (혼재)

uScanApp.h
├── m_StructFaiMeasure[][][][] (전역)
├── m_bFAIUse[] (전역)
```

신규: FAI/ 폴더 분리

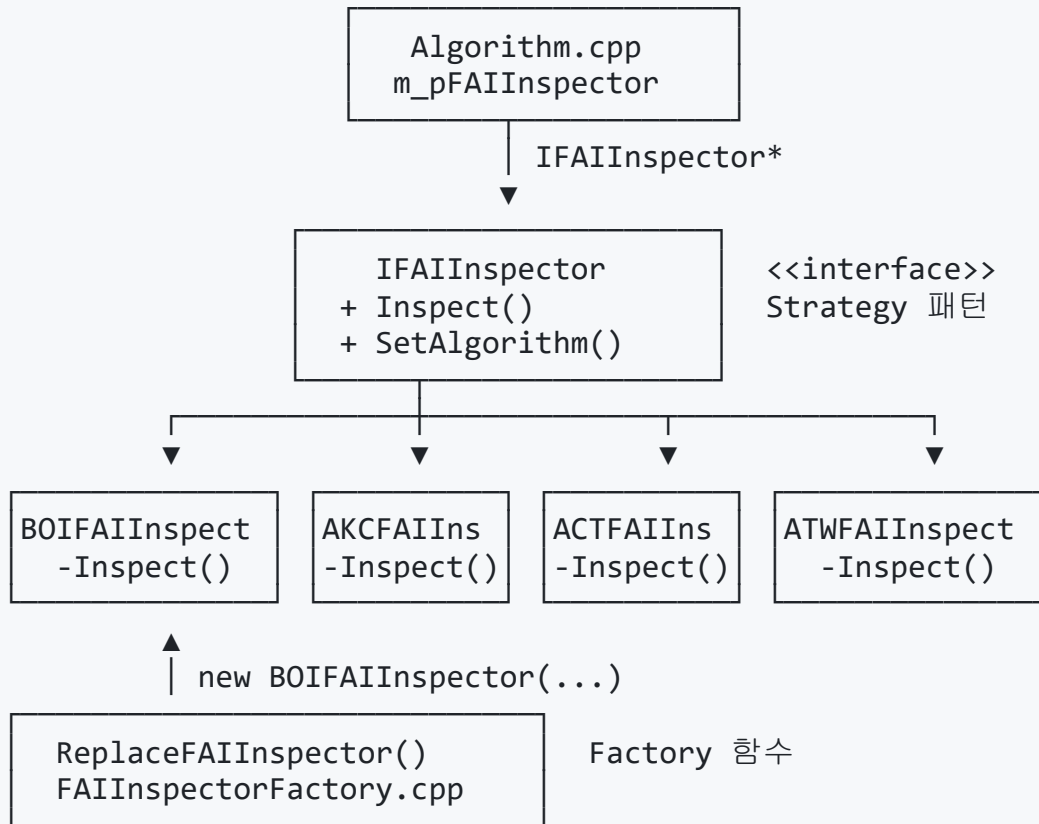
```
FAI/
├── FAIDefine.h ← 공통 상수/enum (측정포지션)
├── IFAIInspector.h ← 모델 인터페이스
├── FAIInspectorFactory.cpp ← 팩토리 (모델 변경 시 처리)
├── FAIMeasureSpec.h ← 측정 명세 (미 구현)
├── FAIMeasureRegistry.h ← 명세 레지스트리 (미 구현)
├── ModelData/
│   ├── FAIDataManager.h ← 데이터 관리(싱글턴)
│   ├── FAIMeasureData.h ← CenterlineMeasureStruct
│   └── FAIModelConfig.h ← 모델별 설정
├── ModelTypes/
│   ├── BOIFAIInspector.h/cpp ← BOI FAI 로직
│   ├── AKCFAIInspector.h/cpp ← AKC FAI 로직
│   ├── ACTFAIInspector.h/cpp ← ACT FAI 로직
│   └── ATWFAIInspector.h/cpp ← ATW FAI 로직
```

핵심 신규 파일

파일	역할
FAI/FAIDefine.h	전체 FAI 공통 상수, <code>enMeasurePos</code> enum, <code>MtoI()</code> / <code>ItoM()</code> 변환 함수, 모델별 측정 위치 범위 상수
FAI/ModelData/FAIModelConfig.h	모델별 측정 포지션·FAI 항목 설정 구조체, <code>Create[모델명]ModelConfig()</code> 팩토리 함수 선언, LOT 헤더 인덱스 (<code>LOT_HEADER_INDEX</code>), 모델별 FAI 번호 namespace (<code>B0I_FAI::</code> , <code>AKC_FAI::</code> 등)
FAI/IFAIInspector.h	FAI 검사 인터페이스 (Strategy). <code>Inspect()</code> , <code>SetAlgorithm()</code> 순수 가상 함수 선언 + <code>ReplaceFAIInspector()</code> 팩토리 함수 선언
FAI/FAIInspectorFactory.cpp	<code>ReplaceFAIInspector()</code> 구현. 모델 타입에 따라 올바른 Inspector를 생성
FAI/FAIMeasureSpec.h	FAI 항목 하나의 측정 명세 (<code>FAIMeasureSpec</code> 구조체): 측정 타입, 허용값, 의존 FAI ID
FAI/FAIMeasureRegistry.h	<code>FAIMeasureSpec</code> 들을 <code>faiId → spec</code> 맵으로 관리하는 싱글턴 레지스트리
FAI/ModelData/FAIDataManager.h	FAI 측정 데이터 및 Use 플래그를 관리하는 싱글턴 (<code>CFAIDataManager</code>). 기존 앱 전역 배열을 캡슐화
FAI/ModelData/FAIMeasureData.h	<code>CenterlineMeasureStruct</code> 정의 (측정 좌표, 결과값 저장)

참고: `FAIItemDescriptor` 의 측정 위치 필드는 단일 `enMeasurePos` 값으로 관리한다.
 측정 위치가 없는 항목은 `MEASURE_POSITION_ETC` 를 기본값으로 사용한다.

아키텍처: Strategy + Factory 패턴



Algorithm.cpp 는 IFAIInspector* 만 알고 있음. 모델별 구현 세부 사항을 모름 → 모델 추가 시 Algorithm.cpp 수정 불필요

코드 비교

코드 비교 ①: 측정 데이터 접근

기존 방식

```
// Algorithm.cpp 내부 어딘가...
// 전역 앱 객체에 직접 접근
CuScanApp* TEHAPP = (CuScanApp*)AfxGetApp();

TEHAPP->m_StructFaiMeasure
    [iMzNo-1][iTrayNo-1][iModuleNo-1][iFAIParam]
    .m_dFAIMeasureValue[5][0] = dResult;

// Teaching 모드/Auto 모드 분기를
// 호출 측에서 직접 처리
if (bIsAutoInsp)
    pStruct = TEHAPP->m_StructFaiMeasure[...];
else
    pStruct = TEHAPP->m_StructTeachingFaiMeasure;
```

문제: 배열 인덱스 범위 검사 없음. 낮은 가독성

새 방식

[illegible]

장점: 범위 검사 내장, 모드 분기 캡슐화, 의미 있는 상수명

코드 비교 ②: FAI Use 플래그

기존 방식

```
// 사용 여부 확인
CuScanApp* pApp = (CuScanApp*)AfxGetApp();

if (pApp->m_bFAIUse[5])
{
    // FAI-5 처리
}

if (pApp->m_bMeasurePosFAIUse[5][3])
{
    // FAI-5의 측정 포지션 3 처리
}

// 설정
pApp->m_bFAIUse[5] = TRUE;
```

문제: "5"가 무엇인지 불명확, 앱 전역 접근

새 방식

```
auto& dm = CFAIDataManager::GetInstance();

// FAI Use 확인 (named constant 사용)
if (dm.IsFAIUsed(BOI_FAI::LENS_INNER_DIAMETER))
{
    // FAI-5(Lens Inner Diameter) 처리
}

// 측정 포지션별 Use 확인
if (dm.IsFAIUsedAtPos(
    BOI_FAI::LENS_INNER_DIAMETER,
    enMeasurePos::MEASURE_TOP_LENS_INNER))
{
    // Top Lens Inner 포지션 처리
}

// FAI Use Flag 설정
dm.SetFAIUse(BOI_FAI::LENS_INNER_DIAMETER, TRUE);
```

장점: 명시적 의미, 캡슐화, 타입 안전 (enum class)

새 모델 추가 시 가이드

1. 체크리스트

- 새 모델 추가 시 체크해야 할 사항

단 계	작업	관련 파일
1	ModelNameCode.ini 에 모델명([MODEL TYPE NAME]) 및 LOT 코드([MODEL TYPE LOT CODE]) 추가 — 해당 파일을 ReadModelNameCodeINI() 가 런타임에 로드	Data/ModelNameCode.ini
2	FAIModelConfig.h 에 새 모델의 FAI 번호 namespace 추가 (xxx_FAI:: 형태)	FAI/ModelData/FAIModelConfig.h
3	enMeasurePos 에 새 모델 전용 측정 포지션 추가 (필요 시)	FAI/FAIDefine.h
4	FAIModelConfig.h 에 Create[모델명]ModelConfig() 선언 추가	FAI/ModelData/FAIModelConfig.h
5	FAIModelConfig.cpp 에 Create[모델명]ModelConfig() 구현 (측정 포지션 목록, FAI 항목 목록, LOT 헤더)	FAI/ModelData/FAIModelConfig.cpp
6	FAIDataManager.cpp 의 Initialize() if-else 체인에 새 모델 케이스 추가	FAI/ModelData/FAIDataManager.cpp
7	FAIInspectorFactory.cpp 의 ReplaceFAIInspector() if-else 체인에 케이스 추가	FAI/FAIInspectorFactory.cpp
8	[모델명]FAIInspector.h/cpp 새 파일 생성 및 FAI 계산 구현(IFAIInspector 상속)	FAI/ModelTypes/
9	uScan.vcxproj 에 신규 .cpp 파일 추가	uScan.vcxproj

2. 신규 모델 등록: ModelNameCode.ini 단일 소스 관리

INI 파일로 모델 타입을 런타임에 관리

ModelNameCode.ini에 추가 (예시 : CHS-X)

```
; Data/ModelNameCode.ini
[MODEL TYPE NAME]
Model_00=CHS-W
Model_01=CHS-Z
Model_02=AKC
Model_03=BOI
Model_04=ACT
Model_05=ATW-D
Model_06=BOS
Model_07=CHS-X      ← 신규 모델명 추가

[MODEL TYPE LOT CODE]
Model_00=0F
Model_01=0H
...
Model_07=XX          ← LOT 코드 추가 (필요 시)
```

모델 식별 방식 (런타임 문자열 비교)

```
// 모델 식별: g_strModelTypeName[] 문자열 비교
CString strModel =
    THEAPP.g_strModelTypeName[THEAPP.GetModelType()];

if (strModel == _T("BOI"))
{
    // BOI 처리
}
else if (strModel == _T("CHS-X"))
{
    // 신규 모델 처리
}
```

동작 원리: 프로그램 시작 시 ReadModelNameCodeINI() 가 INI에서 모델명·LOT 코드를 읽어 g_strModelTypeName[] / g_strModelTypeLotCode[] 를 채움.
모든 모델 분기는 CString 문자열 비교(== _T("..."))로 수행됨.

MODEL_TYPE_MAX = 20 고정: UI 모델 선택 다이얼로그(IDD_SELECT_EQUIP_MODEL_TYPE_DLG)의 버튼이 최대 20개로 제한되어 있으므로 20으로 고정.

Inspector 클래스 구현 예시

새 Inspector 클래스 만들기 (CHS 모델 예시)

CHSFAIInspector.h

```
// FAI/ModelTypes/CHSFAIInspector.h
#pragma once
#include "IFAIInspector.h"
#include "FAIDefine.h"

class Algorithm;

class CHSFAIInspector : public IFAIInspector
{
public:
    explicit CHSFAIInspector(Algorithm* pAlgorithm);
    virtual ~CHSFAIInspector() override = default;
    virtual void SetAlgorithm(Algorithm* pAlgorithm) override;
    // 반드시 구현해야 하는 순수 가상 함수
    virtual void Inspect(BOOL bIsAutoInsp,
                        int iInspectionBufferIdx,
                        int iFAIParamNo) override;
private:
    Algorithm* m_pAlgorithm;
    FAIMeasureRegistry& m_registry; // 측정 명세 레지스트리 참조 (현재 미구현. AKC 참고하여 기존과 같이 구현해야 함)
};
```

CHSFAIInspector.cpp

```
// FAI/ModelTypes/CHSFAIInspector.cpp
CHSFAIInspector::CHSFAIInspector(Algorithm* pAlgorithm)
    : m_pAlgorithm(pAlgorithm)
    , m_registry(FAIMeasureRegistry::GetInstance())
{
}

void CHSFAIInspector::Inspect(BOOL bIsAutoInsp,
                              int iInspectionBufferIdx,
                              int iFAIParmNo)
{
    auto& dm = CFAIDataManager::GetInstance();
    CenterlineMeasureStruct* pStruct =
        dm.GetMeasurePtr(bIsAutoInsp, /*iMz*/1, /*iTray*/1,
                        /*iModule*/1, iFAIParmNo);
    if (!pStruct) return;

    // CHS 전용 FAI 계산 로직 구현
    // ...
}
```

팩토리 코드 작업 방법

팩토리 등록 방법 (모델 전환 코드)

FAIInspectorFactory.cpp 에 if-else 분기 추가

```
// FAI/FAIInspectorFactory.cpp
#include "IFAIInspector.h"
// ....
#include "CHSFAIInspector.h"    // ← 1. 새 헤더 include
#include "FAIDataManager.h"
#include "Define.h"

// 시그니처: enum 대신 CString 문자열로 모델 식별
void ReplaceFAIInspector(IFAIInspector*& pCurrentInspector,
                        CString strModelType,
                        Algorithm* pAlgorithm)
{
    delete pCurrentInspector;
    pCurrentInspector = nullptr;

    if (strModelType == _T("BOI"))    pCurrentInspector = new BOIFAIInspector(pAlgorithm);
    else if (strModelType == _T("ACT"))    pCurrentInspector = new ACTFAIInspector(pAlgorithm);
    else if (strModelType == _T("AKC"))    pCurrentInspector = new AKCFAIInspector(pAlgorithm);
    else if (strModelType == _T("ATW-D"))    pCurrentInspector = new ATWFAIInspector(pAlgorithm);
    else if (strModelType == _T("BOS"))    pCurrentInspector = new BOSFAIInspector(pAlgorithm);
    else if (strModelType == _T("CHS-X"))    pCurrentInspector = new CHSFAIInspector(pAlgorithm); // ← 2. 신규 추가
    else
    {
        TRACE("Unknown model type: %s. FAI Inspector not created.\n", strModelType);
    }
}
```

그 외

동적 FAI 측정 기능 (예정)

별개 문서 참조: 동적 FAI 측정 기능의 상세 아키텍처, INI 포맷, 구현 TODO는

[동적FAI_개발_계획서.md](#) 를 참조하십시오.

요약

- P5(점-점 거리/내접원 반경), P7(선-선 각도) 측정 항목을 **소스 수정/빌드 없이** 런타임에 추가·변경
- 규칙은 `Data/FaiDynamicRules_{Model}.ini` 파일로 관리
- 각 Inspector의 `Inspect()` 마지막에서 `FAIDynamicRuleManager::ExecuteP5Rules()` / `ExecuteP7Rules()` 호출
- UI 진입: 각 `TeachFai*Dlg` → `IDC_BTN_OPEN_DYNAMIC_RULE_MGR` 버튼 → `CTeachFaiDynamicRuleDlg`

정리

핵심 요약

1. 데이터 접근은 `CFAIDDataManager::GetInstance()` 를 통해 — FAI 관련 데이터 한 곳에서 관리
2. FAI 번호는 `BOI_FAI::` , `AKC_FAI::` 등 named constant 사용 — 매직넘버 사용 자제. namespace는 `FAIModelConfig.h` 에 정의됨
3. 새 모델 추가는 9단계 체크리스트를 따라서 — `Algorithm.cpp` 수정 없이 Inspector 파일만 추가
4. 동적 FAI 측정 기능은 아직 미구현 — 빌드 없이 INI 파일로 규칙 추가/변경 → 상세는 [동적FAI_개발_계획서.md](#) 참조

관련 문서

문서	경로
브랜치 목표	FAI/Docs/1_FAI_리팩토링_브랜치_목표.md
측정 패턴 정리	FAI/Docs/2_0_FAI_측정 패턴_정리.md
조건부 컴파일 정리	FAI/Docs/3_조건부컴파일_정리.md

문의

코드 리뷰 또는 구현 관련 문의: 비전 SW / 김준호 선임(연)

개정 내역

날짜	작성자	내역	비고
2026/03/04	비전SW 김준호	초안 작성	
2026/03/05	비전SW 김준호	모델 타입 관리 구조 변경 반영 (ModelTypeList.h + ModelNameCode.ini)	
2026/03/10	비전SW 김준호	모델별 FAI 번호 namespace (BOI_FAI 등) 및 LOT_HEADER_INDEX 를 FAIDefine.h 에서 FAIModelConfig.h 로 이동 (응집도 향상)	
2026/03/11	비전SW 김준호	FAIItemDescriptor::measurePosition 을 단일 enMeasurePos 로 정리하고, 위치 없는 항목은 MEASURE_POSITION_NONE 으로 처리하도록 반영	
2026/03/16	비전SW 김준호	동적 FAI P5/P7 측정 엔진 추가 — FAI/Dynamic/ 신규, 각 Inspector에 Execute 호출 삽입, TeachFaiDynamicRuleDlg 스켈레톤 생성 (UI 구현 TODO 잔여)	
2026/03/16	비전SW 김준호	동적 FAI 측정 엔진 섹션을 동적FAI_개발_계획서.md 로 분리, 본 문서에는 요약 및 참조 링크만 유지	
2026/03/24	비전SW 김준호	ModelTypeList.h 삭제 및 enMODEL_TYPE 제거에 따른 모델 관리 일원화 반영 — 신규 모델 추가 절차를 INI 런타임 로드 방식으로 현행화, 팩토리 코드 예시를 switch(enum) → if-else(CString) 로 교체, MODEL_TYPE_MAX=20 고정 배경 설명 추가	